

BLUE JAY NIGHT RIDE · TEAM TECHNICAL GUIDE

A practical guide to dispatch optimization with aggregate data

Version 2.1, 30 September 2026: from April aggregates to a playable dispatch lab

Research question

Can coordinating assignments across pending requests serve more passengers within pickup and ride-time limits, using the same fleet?

One-line takeaway

We aren't recreating April. We mine it for what it can support, build an April-shaped arena, play five dispatchers on identical nights, and measure how much of the gap to perfect each one closes.

Our only operational source is the April 2026 aggregate report. We digitize every chart, mine it for insights and operating regimes, build an April-shaped base scenario that grows from one van, compare five dispatchers under identical conditions, and identify when coordination helps or hurts. We are not reconstructing individual historical trips, and we are not recreating April.

What this document is, and is not. Research design, implementation instructions, data contracts and a value-entry workbook for Team FourSight: a proposed specification for team discussion. Its April numbers are confirmed Phase 1-2 results from the evidence pack in docs/evidence/, each with its label. There are no verified service rules, simulation code or results yet, and pilot, ASSUMED or provisional values are not Hopkins facts.

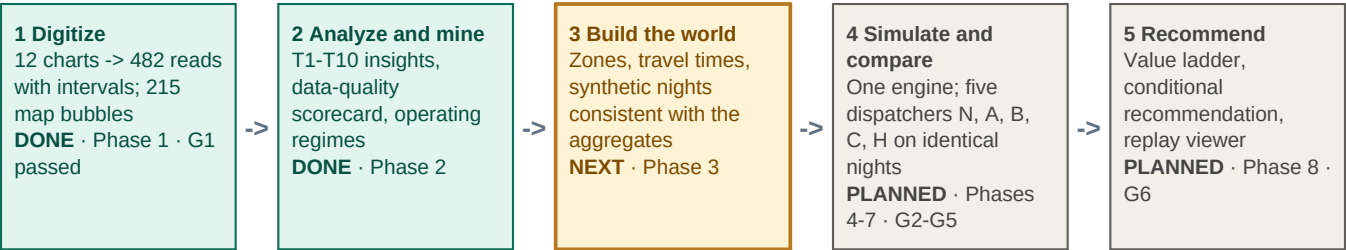
Q	Four questions, revised in v2	Change from v1	Ch.
Q1	What does April actually say?	Every chart digitized first	4, 6
Q2	Can we build an April-shaped base scenario and a simulator we trust?	Changed: no longer "reproduce April"	7, 8, 10
Q3	With everything else held fixed, does coordination beat greedy? When? How close does each policy get to a perfect-information ceiling?	Sharpened	9, 11
Q4	What should Hopkins and TransLoc do or measure?	Same	5, 12

How to read this guide. Read Part I together first: Phase 0 ends when every teammate can explain chapter 2 and chapters 7-9. Build Part II in order; each chapter ends with its gate, and no policy result is interpreted before the gates pass. Look up Part III as needed.

Part I, Think (read first)	Part II, Build (in order)	Part III, Reference
1 The project in one page, p. 2	6 Evidence (G1), p. 8	A Data contracts and files, p. 20
2 What's hard and what isn't, p. 3	7 Zones, generator, base scenario, p. 10	B Parameters and decision log, p. 22
3 Truth, anchors, synthetic data, p. 4	8 The game engine (G2), p. 12	C Workflow, agents, notebooks, p. 23
4 What April tells us, p. 5	9 Dispatchers and the MILP (G3), p. 14	D Sources, risks, AI disclosure, p. 26
5 Value to Hopkins and TransLoc, p. 7	10 Calibration-lite (G4), p. 16	E What more is possible, p. 28
	11 Experiments and value (G5), p. 18	F Revision history, p. 29
	12 Viewer and recommendation (G6), p. 19	What moved where (v1 to v2), p. 30

PART I · THINK · 01

The project in one page



Stages 1 and 2 are the data-mining core. Mining the aggregates for sufficient statistics and operating regimes (supply-limited hours, a congestion regime, the Apr 17 outlier) is what keeps stage 3 honest: the synthetic nights must be consistent with everything stages 1-2 established. This is the sequence Prof. Kearsley sketched in his first note (Reference C).

The operational decision

At each dispatch event, which vehicle should serve each waiting request, and in what pickup/drop-off order? A second decision is whether to wait briefly for additional requests before committing. We will distinguish assignment quality from the effect of waiting. The player ladder in chapter 9 separates the value of pooling, waiting, coordination and information, one ingredient at a time.

What counts as success

A useful outcome may be improved service, a rule identifying when coordination is worthwhile, or evidence that a simpler method performs just as well. A positive improvement is not guaranteed, and a null result is still a result (chapter 11). The practical deliverable is a candidate policy and its failure conditions, suitable for consideration in a future operational pilot.

In scope	Outside the initial scope
Aggregate audit of all 12 April charts; broad geographic zones; real road costs	Recovery of actual rider histories or the incumbent policy
An April-shaped base scenario plus several demand, fleet and travel-time variants	Recreating April night by night; a single supposedly correct synthetic history
Pooling-capable baselines, joint assignment and an offline ceiling (H)	Building-by-building student activity reconstruction
Passenger service, modeled overflow, computational feasibility	Guaranteed Lyft savings, real vehicle control, deployment, replacing TransLoc's dispatcher
Checks on spatial service differences	Demographic fairness claims without demographic evidence

Four evidence labels used throughout

Label	Meaning and example
OBSERVED	Read from a source with its definition and extraction uncertainty.
DERIVED	Computed from compatible observations, e.g. passengers per completed request.
ASSUMED	Chosen for a scenario, e.g. campus departures are stronger early.
SIMULATED	Produced by the model, e.g. waiting time under policy B.

External geography and road estimates receive their own source and retrieval date. They improve geographic or travel-time realism; they do not establish where students requested trips at a particular hour.

Where we started, and the directions from here

Line	Direction	Status
0	Initial scope (proposal, 15 Sep): replay April trip by trip and optimize it with one node-arc MILP.	ABANDONED 28 Sep: the trip-level data never came. Salvaged: the node-arc MILP returns as H, the ceiling (chapter 9); "replay" returns as the viewer, which replays simulated nights (chapter 12).
1	Simulation and dispatch (the project): an April-shaped arena, five dispatchers on identical nights, the value ladder.	ACTIVE: stages 3-5 above.
2	Regimes and the calendar: Apr 17, the opening night of the 150th All-Alumni Weekend (with a JFX lane closure), was a slow night, not a busy one. A calendar might predict such nights (campus events, road closures, weather, holidays): an event-aware forecast of daily load and slowdowns, scored out of sample as new monthly reports arrive.	SIDE-BRANCH, time-boxed: a brief only, after G2. One month holds one outlier night, so April alone cannot estimate an event effect ($n = 1$).
3	What we do not see: the direction in plain sight but outside our frame.	OPEN: Prof. Kearsley's view invited in our reply to his 29 Sep note.

PART I · THINK · 02

What's hard and what isn't

NP-hardness limits what we can prove optimal. It does not limit what we can simulate.

A simulator only plays events forward. TransLoc's real dispatcher also runs every night on fast heuristics and never solves the NP-hard problem exactly. The real obstacle for this project is data, not complexity.

Task	Hard?	Why	How we handle it
Simulate one night with a greedy dispatcher	No	Polynomial; seconds	n/a
Simulate all of April	Cheap to compute, but not worth doing	We have no night-level inputs to replay	We don't. The reason is the data, not complexity: we build an April-shaped base scenario instead (chapter 7)
Policy C: an MILP at each tick	A small NP-hard subproblem	Only pending requests (~5-15 at peak) × a capped set of candidate routes	Expected well under a second per tick, and logged. The rolling horizon is the answer to NP-hardness
H: the offline optimum for a whole hour	Genuinely hard	The full dial-a-ride problem (DARP) with time windows	Small instances only. If the solve doesn't close, the solver's bound is still a valid ceiling, so we report the bracket [incumbent, bound]
Calibration	No	A 2-4 parameter search using the cheap policy A. Not combinatorial	A grid of ~100-200 runs × a few seconds each (chapter 10)
Synthetic demand	Trivial to compute	It's sampling. The hard part is identification: many O-D tables fit the same margins	Several labelled scenarios (chapter 3)

What this means for the build: simulate freely, solve exactly only where the problem is small (S1, S2), and report a bracket everywhere else.

PART I · THINK · 03

Truth, anchors and synthetic data

There is no ground truth for the answer. But the assumptions are fenced, and there are three kinds of statement:

Kind of statement	What it covers	Rule	Label
Hard fences	Reconciled aggregates, e.g. the city maps (O 24,056, D 24,057 completed rides) match the hourly (24,016) and daily (24,095) completed totals within 0.4% (chapter 4)	A scenario that breaks a fence is wrong	OBSERVED, DERIVED
Labelled choices	Anything the data doesn't pin down: the O-D joint distribution, direction by hour, patience, capacity, and limits	Varied as scenarios, never presented as Hopkins facts	ASSUMED
Exact truth inside the simulator	Every request is known	Policy comparisons are exact and paired	SIMULATED

Anchors are values printed on the April charts: tooltips and data labels (listed in chapter 4). They check every extraction (chapter 6). The load anchor, completed rides per van-hour at 6 PM, sets how hard the base scenario is (chapter 7).

Is synthetic data possible? Yes, as plausible streams, not as truth

It is standard trip distribution from transport planning: gravity weights × IPF (iterative proportional fitting) to hit the O and D margins, then sampling. Drawing random points from the density clusters is the sampling step. Many O-D tables fit the same margins, so we run several labelled variants: month-average, campus-outbound at 6 PM, and bursty at 18:00.

PART I · THINK · 04

What April tells us

Status: Phases 1-2 are complete. The numbers in this chapter are confirmed results from the evidence pack in docs/evidence/ (README.md, data_quality.md, reconciliation_ledger.md), each with its evidence label. They replace the preliminary eye-read of plan v2. Use the pack, not this page, for any calculation.

The 12 charts on 11 pages, as digitized (briefs in docs/evidence/briefs/)

p	Chart	Check against the printed values	Quirks and definition gaps
2-5	O/D density maps (city and campus views)	Two blind readers agree on 214 of 215 bubbles; city O 24,056 vs D 24,057	Campus edges cut (4 O and 8 D bubbles with bounds only); one partly hidden city label (read 6036, bounds 6030-6039)
6	Average vans in service by hour	27.56 vs 27.6	"All services", averaged over 29 days, so per-van rates are bounds
7	Rides by status, daily	No anchor; three reading methods agree within 0.6 px	"Canceled" is undefined; "many cancelled become Lyfts" is a Hopkins annotation
8	Passengers, daily	1,387.4 vs 1,388	Van riders only (matches the p10 van series, r 0.9994)
9	Completed rides by hour	5,089 vs 5,098	The hour may be request, pickup or completion time (unknown)
10	Van + Lyft riders (PowerBI)	44,248 vs 44,374 printed (-0.3%)	Low resolution (±28 riders per bar); the day-of-week axis lists Friday before Thursday; the labels themselves are correct
11	Wait P10, median, P90 (daily)	Apr 17 anchors within 4 s	Min and Max hidden; rider vs request weighting unknown
12	Ride-time P10, median, P90 (daily)	Apr 17 anchors within 4 s	Three quantiles per day, not a distribution; not by hour

Data quality: the six-dimension scorecard

docs/evidence/data_quality.md scores each chart High, Medium or Low on six dimensions: readability, consistency, definition, resolution, coverage and usefulness.
Readability and consistency are high. Reads reproduce the printed values within about 1%, and independent charts agree within 0.4% on the same totals. Extraction is not the weak link.

Definitions are the weak link: what "Canceled" includes, the "All services" vehicle filter, and the hour attribution of rides. Better reading cannot fix them, so each is carried as a stated alternative (the G1 caveats, chapter 6).

Resolution caps what we can claim: monthly space and daily or hourly time, never both together. Where demand occurs at a given hour, or where unserved requests came from, is outside what this data can show; hence April-shaped scenarios, not reconstructed nights.

Coverage is high (the vans chart covers 29 days; campus map edges are cut), and every chart serves as an input, a calibration target or a check.

Finding: the maps count completed rides (D-01)

Four independent charts land on the same total:

Chart	Total	Evidence
City origin bubbles (p2)	24,056	OBSERVED: sum of 21 bubble reads
City destination bubbles (p3)	24,057	OBSERVED: sum of 22 bubble reads
Hourly completed rides (p9)	24,016	OBSERVED: sum of the hourly bars (maps +0.16%)
Daily completed rides (p7)	24,095	OBSERVED: sum of 30 daily reads (hourly -0.33%)

So the maps count completed rides (D-01, DERIVED). They do not count requests (50,297 by status sum, T2) or passengers (33,394). The origin-destination residual of 1 ride sits inside the reading range of the one partly hidden label: it is carried, not forced. The spatial margins describe served trips, and where the cancelled requests came from is unobserved. The v1 guide warned this was possible; the data now shows it.

Calendar day versus service night (D-03, ASSUMED). The charts use calendar days, so each service night is split at midnight. Apr 30 is 187 completed rides below the mean day, and after-midnight rides average 108 a night: the Apr 30 night ends in May, which explains part of the gap. The rest is retained in the ledger.

The ten insight tasks (Phase 2): confirmed findings

T	Finding	Label	What it cannot tell us
T1	Supply-limited: completed rides track the fleet hour by hour (4.8-6.2 rides per van-hour), and daily completions do not respond to requests (slope -0.02, r -0.04).	DERIVED	Demand by hour; whether more vans would turn cancellations into rides one-for-one.
T2	47.4% of 50,297 April requests were cancelled; cancellations outnumbered completions on 13 of 30 days; 2.09 requests per completed ride.	DERIVED	Cancellations by hour; why requests were cancelled; how many were repeats.
T3	Cancelled share rises ~7 points per extra minute of median wait (r 0.74; 0.72 within weekday). Apr 17 (Fri) is the outlier: total riders exactly at the month's median but the fewest van completions (543), the most Lyft riders (674) and the longest waits and rides. The system slowed; demand did not spike.	DERIVED, associational	Individual patience; hour-of-night effects; causal direction; why Apr 17 slowed: it was the opening night of the 150th All-Alumni Weekend, with a southbound JFX lane closure and a dry evening (docs/evidence/context/april_2026_calendar.md), but one night cannot separate event, closure and an ordinary Friday.

T	Finding	Label	What it cannot tell us
T4	1.39 passengers per completed ride (1.21-1.63 by day), higher at weekends (1.49 vs 1.35); no sign that it changes with volume.	Mean DERIVED; shape ASSUMED	The party-size distribution (solo riders, pairs, groups).
T5	Little's law at 6 pm: ≥ 1.31 rides in progress per van (≥ 1.81 riders), so pooling is certain at the peak; later hours are marginal (1.02-1.18). Ride time fits a lognormal (mean 12.7 min); waits do not.	DERIVED (lower bounds)	How often a van carries 3+ riders; capacity binding; empty driving.
T6	Rides take 2.2-2.7× the direct drive; the P90 ride (23.9 min) exceeds the longest direct drive (13.6 min), so the tail comes from routing.	DERIVED; O-D pairing ASSUMED	The true O-D pairing; how the excess splits into dwell, slow driving and pooling detours.
T7	Lyft carries 24.5% of riders (10,821 of 44,248; 9.9-46.3% by day); daily Lyft riders track cancellations (r 0.88).	DERIVED, associational	How many cancellations became Lyft trips (no hand-off is recorded); cost.
T8	About 80% of trips are in the Homewood area. The campus core and the Charles/33rd corner are net sources; University Pkwy and Remington are net sinks.	DERIVED	Direction by hour; where cancelled requests came from; individual O-D pairs.
T9	6 pm carries 21.2% of rides on 17.5% of van-hours; 2 am has 4.53 vans (2.9% of van-hours) and ~0 completed rides; 7 pm to 1 am is proportional.	DERIVED	Whether the peak is under-supplied; which vans run at 2 am.
T10	Cross-chart totals close within 0.4%; the campus-frame and Apr 30 gaps are explained or retained. G1 passed with 7 carried caveats (chapter 6).	DERIVED	What the charts leave undefined: those caveats stay as alternative interpretations.

Phase 2 exit check: each number traces to observation IDs with an evidence label; each figure has a "cannot tell us" line. Briefs: docs/evidence/insights/tNN.md; figures and CSV tables: outputs/insights/.

PART I · THINK · 05

Value to Hopkins and TransLoc

Hopkins Transportation is the client and TransLoc is the vendor. TransLoc already runs a real-time dispatcher, so what we offer them is evidence, not a replacement dispatcher.

What we offer	Why it matters
1. A batching and coordination benchmark for this service, including when it hurts	The window W is a configuration lever.
2. Headroom (H - C)	If it is large, the next lever is prediction and pre-positioning, which TransLoc could build.
3. Fleet and shift curves	Service level versus vans by hour. T1 and T9 suggest this may be the biggest lever.

What we offer	Why it matters
4. A cancellation diagnosis	47.4% of requests were cancelled (DERIVED, T2). The congestion and patience curves (T3: ~7 points per extra minute of median wait) show how much the wait must fall to cut cancellations and Lyft cost.
5. The value of the next extract	Every ASSUMED value in our register corresponds to a field TransLoc already records. The cheapest five (Reference E) would pin the base load (30 or 60 requests: today a 2x range on our main difficulty knob), the hourly demand curve and the O-D pairing. The project finishes without them; they are what would turn plausible scenarios into validated ones.
6. The replay viewer	A communication tool for Hopkins stakeholders (chapter 12).

Headline discipline: every headline starts "Under these tested scenarios and assumptions..."

PART II · BUILD · 06

Evidence: digitize and reconcile

Create a source manifest first: one chart brief per chart (fields below). For each chart, record title, PDF page, reporting period, unit, population/status filter, aggregation grain and anything unknown. Do not rely on numbers quoted in earlier conversations; the evidence pack in docs/evidence/ is the record.

Chart family	Enter into	Checks before downstream use
Daily requests by status	observations table: total/completed/cancelled/no-show/denied	Are categories exclusive? Are totals given or reconstructed? Can repeat bookings occur?
Hourly completed rides	observations: hour and completed count	Determine whether the hour refers to request, pickup or completion. Unknown remains unknown.
Origin/destination maps	cluster observations and zone allocations	Read counts; assess crop/occlusion and positional uncertainty. Do not assume each marker is a building.
Passengers; van/Lyft summaries	observations with source-system tags	Check trips versus passengers and dates before forming ratios.
Fleet by hour	observations with service coverage	Resolve or scenario-test all-services versus Homewood coverage and reporting days.
Wait/ride summaries	observations with quantile and unit	Check served population and passenger/request weighting. Do not infer a full distribution from a few quantiles.

What the PDF gives us

Verified in the PDF: every chart is a raster screenshot, so there are no vector data to extract.

Charts	Format and precision
Maps (pp. 2-5)	JPEG, ~91-107 ppi.
Highcharts panels (pp. 6-9, 11-12)	Lossless, ~101-106 ppi, using Highcharts' default palette. That makes color-mask extraction precise.
PowerBI panel (p. 10)	Only 776 × 232 px at 62 ppi, so it will be low precision.

Extraction procedure

Bars and lines

1. Run `pdftimages -png` to get the original pixels.
2. Calibrate the axes and units from gridlines and tick labels.
3. Mask each series by its Highcharts color (bars: top edge; lines: marker centroids).
4. Read each value with an interval of $\pm(1 \text{ px} + \text{marker radius})$ in data units.
5. Reproduce the visible tooltip anchors.
6. Make an overlay image: extracted points drawn on the original chart.
7. Make a clean recreated chart.
8. Do an independent second read on a sample, including peaks and anomalies.

Maps

1. Georeference each screenshot. Google Maps is Web Mercator at a fixed zoom, so a scale-plus-offset fit from 4-6 ground-control points (labelled POIs and intersections) is enough. Target error is ~100 m; zones only need 300-500 m.
 2. For each cluster, record the pixel center, the count read by eye, and its color bucket. The color bucket is a free check on every reading: blue = 1 digit, yellow = 2, red = 3, magenta = 4.
 3. Flag cut-off bubbles with bounds.
 4. Reconcile campus clusters against city clusters without double counting.
- Outputs: a cluster table and a GeoJSON file, plus a digital O/D map with circles sized by count, an O/D toggle, and an opacity slider that overlays the original screenshot for QA.

All charts: preserve the original observations when later reconciliation changes interpretation.

Chart brief: one card per chart

Title as shown · page · source system · service filter · date range · grain · definition (quoted subtitle) · units · series · anchors · extraction method and error · quirks · what it can and cannot tell us · which T-tasks use it · evidence label.
 Template: `docs/evidence/brief_template.md`; the filled briefs are in `docs/evidence/briefs/`.

Reconciliation ledger

For each proposed identity, record left-hand total, right-hand total, residual, coverage compatibility and action.
 Examples: daily versus hourly completed totals; passenger series versus van ridership; van plus Lyft versus reported combined total. A discrepancy may be a definition issue, not measurement noise. The ledger is `docs/evidence/reconciliation_ledger.md` (task T10), signed at G1.

Do not force equality: keep an explicit unknown/outside-map component where warranted. Never allocate unexplained residuals to convenient zones just to make the generator balance. If filters remain unknown, model alternative interpretations.

Gate G1, input integrity: PASSED 2026-09-29, with carried caveats.

Required evidence: Definitions and coverage recorded; reconciliation residuals explained or retained; units verified.

Phase 1 checks: tooltip anchors reproduced within tolerance; overlays inspected by eye; the second read agrees within tolerance; the digit-count/color rule holds for every cluster; ledger residuals are explained.

Verdict: every cross-chart identity closes within 0.4%, and two blind map readers agree on 214 of 215 bubbles (docs/evidence/reconciliation_ledger.md). The evidence may be used downstream with the caveats below; it supports no Hopkins-specific claim about demand or dispatch outcomes.

Carried caveats:

1. Vans are "All services" over 29 days, so per-van rates are bounds (T1, T5).
2. The hour attribution in p09 (request, pickup or completion) is unknown.
3. "Canceled" is undefined (rebookings, Lyft handoffs?), so requests bound distinct demand from above (T2, T7).
4. p08 counts van riders only (it matches the PowerBI van series).
5. The maps count completed rides (D-01); where cancelled requests came from is unobserved.
6. The p07 zero line reads +3.3 rides/day on "Denied" (a reading offset, inside the interval).
7. A triangle-marker bias of ~0.5-1 px is documented on p11.

If it fails: Do not combine incompatible series. Carry alternative interpretations.

PART II · BUILD · 07

Zones, generator, base scenario and ladder

Build the world in this order: zones and travel times, then the generator, then the base scenario and its ladder. Steps 1-5 apply to every scenario; the base scenario fixes one April-shaped instance of them.

Step 1: reconcile spatial evidence

Place origins and destinations on a common coarse zone system. Broad and close-up maps may overlap; do not double-count them. Use uncertain allocations when cluster footprints are not known. A map count is not proof of its exact spatial extent or population.

Step 2: create candidate O-D-time weights

For origin o , destination d and hour h , use a positive seed such as:

$$\text{seed}[o,d,h] = \text{base}[o,d] * \exp(-\text{beta} * \text{travel_min}[o,d,h]) * \text{activity_weight}[o,d,h]$$

The activity weight represents an explicitly chosen directional scenario. Compare early campus departures, early campus arrivals and mixed flows. Include feasible intra-zone trips where relevant; coarse zones can contain legitimate journeys. Exclude impossible pairs with documented structural zeros, not merely inconvenient ones.

Step 3: balance compatible margins

For compatible populations, use iterative scaling (IPF) to match origin totals, destination totals and the selected hourly distribution. Stop when residuals meet a declared tolerance. An average-zero directional adjustment alone does not guarantee these margins. Infeasible support must produce a diagnostic, not silent redistribution.

The city maps describe completed trips (confirmed, chapter 4), so their margins constrain baseline served outputs, not automatically all generated requests. We don't know where the cancelled requests came from. Either evaluate

alternative interpretations of that or use the maps as an explicit approximate spatial prior and report baseline mismatch. Label which interpretation each scenario uses.

Step 4: sample request streams

Choose a request count from compatible observations or an explicitly scaled scenario; for the base scenario, use the load anchor below. For multi-hour scenarios, allocate requests to hours using assumed arrival shares. Then draw timestamps under smooth (Poisson) or bursty alternatives. Sample O-D pairs, routable endpoints, party sizes and patience. Use separate reproducible random streams for each component.

Step 5: distinguish population targets from sample variation

A probability table can match target margins while a sampled night differs by chance. For exact synthetic monthly margins, use constrained integer allocation; otherwise report sampling residuals. Do not require every night to reproduce a monthly spatial pattern exactly.

Keep input and outcome apart: observed wait and pooled ride-duration summaries are baseline checks. They are not independent request attributes to draw and then reuse as simulated service results. Road time is not pooled ride time.

The base scenario: 18:00-19:00, 5 vans, about 30 requests

We adopt the team's April-shaped design and anchor it in the data: decisions D-04 to D-08 in docs/technical/decision_log.md, with April numbers from the evidence pack (chapter 4).

Setting	Base value	Basis
Window	18:00-19:00	D-05 (DERIVED): the peak and the first hour of service, with zero rides before 6 PM, so no inherited backlog and no warm-up to model. A burst at 18:00 is plausible and becomes a variant.
Load	30 requests ("as served"); 60 is "as requested"	D-06 (DERIVED): the load anchor below
Fleet	5 vans (D-06), starting at hotspots or a depot	Start positions ASSUMED (D-07)
Endpoints	Drawn from the digitized hotspots, jittered and snapped to roads	Map clusters (chapter 6)
Arrivals	Poisson, plus a bursty variant	ASSUMED, varied as scenarios
Party size	Mean 1.39	Mean DERIVED (T4); the shape is ASSUMED (D-08)
Capacity	{4, 7}	ASSUMED
Limits	20 min wait, 25 min ride	PROVISIONAL (D-04)

Load anchor. The 6 PM hour carries 5,089 completed rides in 30 days (printed 5,098), about 170 per hour with 27.6 vans: 6.16 completed rides per van-hour (T9). So 5 vans × 1 h × 6 ~ 30 requests is "as served". Each completed ride stands for 2.09 requests (T2), so 60-63 requests is "as requested"; D-06 uses 60, the upper end of distinct demand, since cancellations include rebookings and hand-offs. The team's proposed 20-30 requests is now a data-anchored choice, and rho = requests per van-hour is the main difficulty knob (D-06 grid: 4, 6, 9, 12).

30 requests on 5 vans is not a trivial case. Unpooled, each request costs ~14 van-minutes (~8 driving + ~4 deadhead + dwell; back-of-envelope): $30 \times 14 \sim 420$ van-minutes against 300 available. Pooling is required, and that is exactly where dispatchers differ. At $\rho = 4$ every policy serves everyone, and only the waits differ.

The scale ladder: grow from one van

Level	Size	Purpose	Exact optimum?
S0	1 van, 1-2 requests	Engine hand-check	By hand
S1	1 van, 6 requests	MILP checked against brute-force enumeration	Yes
S2	3 vans, 15 requests	First multi-van pooling	Expected yes
M (base)	5 vans, 30 requests	Main experiments + the H ceiling	Bracketed
L	~27 vans, ~170-300 requests	Does the ranking hold at real density?	No (A, B, C only)

Can we really do 30 requests with 5 vans? Simulating it and running C every tick is easy. Only the exact offline H is hard. Specialized branch-and-cut codes in the literature solve static DARP instances with a few dozen requests (cite before reporting); a plain model in HiGHS will manage less. So we measure how far it gets, and report the bracket.

Two caveats. 1. One fixed scenario is for explaining (the demo and hand-checks); many seeded variants are for concluding. 2. Pooling potential grows with density, so M may understate what coordination can gain: we keep one L check.

Phase 3 exit check, scenario integrity.

Required evidence: margins reproduced within the declared tolerance; seeds reproducible; base scenario M frozen and seeded; levels S0 to L defined; scenario grid pre-registered.

If it fails: Fix the generator before building on it. Infeasible support produces a diagnostic, not silent redistribution; report sampling residuals.

PART II · BUILD · 08

The game engine

Model the simulator as a game. The analogy describes the design accurately; it is not only a way to present it.

Game	Our simulator
Level (map)	Scenario: road times, requests (arrival times hidden from players), fleet, rules
Level editor	Scenario builder (chapters 3 and 7)
Engine + referee	Event-driven engine: clock, van movement, rule checks. It rejects illegal commands, which also catches dispatcher bugs
NPC behavior	Rider model: a patience clock drawn once per request (identical for every player), then cancel and Lyft-overflow rules
AI players	Dispatchers: the same controls and information, and no peeking at the future. Only H may, by design

Game	Our simulator
Turn	An event: request, tick, arrival at a stop, patience expiry. It is a turn-based game; the viewer interpolates motion between events
Replay file	The event log. It's deterministic: same seed + level + player gives an identical replay
Leaderboard	Metrics across seeds

Code shape

A single Dispatcher interface with a trigger (on_request, or every W seconds) and decide(observation) -> commands. The engine never knows which player it is running. The engine owns time, rules, and the accounting identity. B and C share the urgent-request fallback and the same reassignment rule. Build N and A first, with a minimal replay player (chapter 12), and run the ladder S0 -> S1 -> S2 -> M.

Event loop

```
advance vehicles to next event time
apply arrivals, shift changes, pickups and dropoffs
update pending requests and remaining route feasibility
invoke dispatch if arrival/tick/urgency rules require it
commit allowed decisions and schedule next events
log state transitions; repeat until the night clears
```

Specify deterministic ordering for simultaneous events. For example, process drop-offs before pickups, then new requests, then patience/deadline checks and dispatch, with explicit equality rules. Use the same ordering for every policy.

Request states

An active request may be pending, assigned or onboard. Each request ultimately has exactly one terminal status: **completed, abandoned, modeled overflow, or unserved at close**. State changes require a reason code. An assigned rider can abandon before pickup if that is the shared patience rule; onboard riders cannot simply disappear.

Outcome definitions

Modeled overflow: an explicit outside-option decision under the chosen fallback rule. A request being infeasible at one instant is not necessarily proof it cannot be served later. Specify whether it remains pending until a deadline, is handed off immediately, or uses a forecast threshold.

Abandonment: a rider leaves before pickup according to the scenario patience model. This is not automatically identical to reported cancellations. Administrative cancellations, duplicates and no-shows may not be represented; document that gap.

Vehicle commitments and end of service

Freeze travel already underway and protect all onboard passengers. Define whether the next stop is frozen or only the current road movement. At shift end, stop assigning new work according to the shared rule and complete accepted service. Continue simulation beyond the last arrival long enough to clear accepted trips; account for residual requests explicitly.

Constraints checked after every route change

Pickup precedes drop-off; each party stays on one vehicle; load remains between zero and capacity; all travel has a valid directed cost; service windows and existing commitments remain feasible. Distinguish projected feasibility at assignment from actual attainment if travel times are stochastic. The referee runs these checks on every dispatcher command and rejects illegal ones.

Accounting identity: generated requests = completed + abandoned + modeled overflow + unserved at close. Check passenger-weighted totals separately. No request may appear in two terminal categories.

Gate G2, simulator correctness.
Required evidence: Hand-solvable cases: one request; compatible pooling; capacity conflict; urgent deadline; unreachable pair; shift end; abandonment.
Phase 4 checks: S0 and S1 hand-checks pass, with expected values derived independently of the engine code; the accounting identity holds on every run; determinism: the same seed gives the same replay hash; the referee rejects illegal commands (negative tests).
If it fails: Fix engine and accounting before comparing policies.

PART II · BUILD · 09

Dispatchers and the MILP

Are A, B and C different algorithms or different strategies? Both, on purpose. A dispatcher is **when** it decides (the trigger) plus **how** it decides (the algorithm). Each rung of the ladder changes exactly one ingredient.

Player	When (trigger)	How (assignment method)	What the gap isolates
N: nearest van (floor)	Each request arrival	Nearest van, no pooling	n/a: the floor
A: immediate greedy	Each request arrival; shared urgent-request rule	Best feasible pickup/drop-off insertion, one request at a time (with pooling)	Value of pooling (A - N)
B: periodic greedy	Every W seconds; shared urgent-request rule	Same insertion rule, processing pending requests oldest first	Value of waiting (B - A; may be negative)
C: periodic joint	Same W and urgent rule as B	Choose compatible routes for multiple vehicles and requests jointly: a joint MILP over all pending requests	Value of coordination (C - B)
H: hindsight (ceiling)	Once, knowing the whole hour	Offline MILP (the proposal's node-arc model)	Headroom left (H - C)

B versus A: tests the effect of periodic coordination with a fixed greedy method. **C versus B:** tests joint assignment at the same window. **C versus A:** measures the total intervention, but cannot alone explain which component helped.

Pilot settings to review, not operational facts

A small pilot can use W = 30, 60 and 120 seconds, with A as a separate immediate policy. Do not call W = 0 the joint method: it is a different algorithm. Add an event-triggered joint policy later only if meaningful uncommitted decisions remain to optimize.

Rules held fixed across policies

Use the same request stream, vehicle shifts, capacities, road model, starting state, service limits, rejection rules and objective priorities. Freeze the same committed movements. Policies may see current state and previously revealed requests, never future arrivals. Give B and C the same eligibility for reconsidering previously assigned but uncommitted requests. Only H sees the future, by design: it is a ceiling, not a dispatcher.

Urgent-request behavior

At every new request and dispatch tick, calculate remaining pickup slack using a feasible insertion estimate. Requests that cannot safely wait until the next tick invoke a shared immediate greedy fallback. The margin is a named parameter. Count override events; frequent overrides may mean the window is operationally ineffective.

First pilot questions

Does each algorithm create legal pooled routes? Does B add waiting without gains? Does C find better combinations than B? How often does the optimizer fall back or time out? Run contrasting scenarios before scaling experiments.

Design gate before coding C: agree on the primary metric, dispatch semantics, reconsideration rules and shared emergency fallback before implementing policy C. Otherwise an apparent gain may be a difference in rules.

C: a pooling-capable joint optimizer

A practical first MILP can select from precomputed feasible route options. Each option is an ordered sequence of pickups and drop-offs for one vehicle, retaining its required stops and adding one or more pending requests. Feasibility is checked by simulating that sequence from the current state.

Notation and model

Let $R(v)$ be the candidate routes for vehicle v . Let $x[v,r]$ be a binary route-selection variable. Let $a[q,v,r]$ indicate whether route r serves pending request q . Let $y[q]$ indicate that q is selected for service now, and $p[q]$ be its party size.

$$\begin{array}{ll} \sum_r x[v,r] = 1 & \text{for each vehicle } v \\ \sum_{v,r} a[q,v,r] * x[v,r] = y[q] & \text{for each request } q \\ x[v,r], y[q] \in \{0,1\} \end{array}$$

Priority 1: maximize $\sum_q p[q] * y[q]$
 Priority 2: minimize added waiting, then added travel
 while preserving the best Priority 1 value

Include a no-new-request route for every vehicle. Existing commitments must be served in every eligible option or fixed explicitly. Unselected requests remain pending unless the common overflow or abandonment rule fires. The myopic objective supports the experiment but does not guarantee maximum service over the entire night.

Construct route options carefully

Enumerate feasible insertions and small combinations of new requests, using a configurable cap and pruning. Include options with multiple new requests on the same vehicle; otherwise the experiment cannot test joint pooling. Record the candidate-generation limits and route counts. Where possible, include the greedy solution as a candidate incumbent.

Objective and fairness controls

Use the same service-first priorities and stable tie-breaking for greedy insertion. If passenger weighting disadvantages single riders, report request-level service rates and consider a separate sensitivity case. Do not change objective weights after seeing which policy wins without marking the analysis exploratory.

Computation is part of the policy

Record solve time, incumbent quality and timeouts. Specify whether computational delay enters simulated time; an idealized zero-delay model must be labeled. A time-limited feasible incumbent may be used; otherwise invoke the documented greedy fallback. Bounds apply only to the restricted candidate model, not to the entire routing problem. Solve time is logged on every tick, and a solve slower than W counts as operationally infeasible.

Prototype warning: ordinary request-to-vehicle Hungarian matching selects at most one new request per vehicle per round. It is not equivalent to this route-combination model.

H: the offline ceiling, reported as a bracket

H is the node-arc MILP from the proposal, solved once offline with the whole hour known in advance. No real dispatcher can see the future, so H is a ceiling, not a policy to deploy; H - C is the headroom left. The problem is the full DARP with time windows, so run H on small instances first (S1, S2), then measure how far it gets on M. If the solve does not close, the solver's bound is still a valid ceiling: report the bracket [incumbent, bound].

For the mid-semester story: the node-arc MILP from the proposal comes back as H, and the route-selection MILP in this guide is C. The shift reads as "offline model = ceiling, online model = dispatcher"; nothing in the proposal is contradicted.

Gate G3, policy parity.

Required evidence: Same inputs, commitments, eligibility, service rules and information; matched B/C windows.

Phase 5 checks: C equals brute-force enumeration on S1, so the MILP is verified on a toy instance before it scales; B and C share the urgent-request fallback.

If it fails: Treat comparisons as invalid until aligned.

PART II · BUILD · 10

Calibration-lite

Calibration-lite makes the scenario April-shaped. It is not NP-hard: it is a small search over a cheap simulator, using policy A only, run at L (Phase 6).

Calibration protocol

Before fitting, list which summaries are calibration targets and which are held-out checks. Fit or choose only a small, declared parameter set. Retain multiple plausible parameter combinations if the data cannot distinguish them. Freeze them before policy comparisons. Apply each retained configuration to every policy.

Split physics from behavior

Calibrate road physics on the low tail. Ride-time P10 (4.2 min, DERIVED from p12) is mostly direct trips, so it pins down the speed factor and dwell time independently of the dispatch policy.

Hold out behavior on the high tail. Ride median and P90, and the wait quantiles, are used only as checks.

Targets and checks at the right grain

Summary	Role	Grain and caveat
Completions per van-hour at 6 PM (6.16, DERIVED, T9)	Calibration target	Hourly: the same grain as our window
Ride-time P10 (4.2 min, DERIVED from p12)	Calibration target (road physics)	Daily
Ride median and P90	Held-out check	Daily
Wait quantiles	Held-out check	Daily and covering the whole night, so a loose band, not a fit
Cancellation share	Check on the patience knob	Daily (p7)
Party size	Input, not a target	Daily ratio (T4)

Knobs, budget and freeze

Knobs (2-4): OSRM speed factor, dwell time, demand scale (at L), and patience (checked against cancellation share).

Budget: for example $5 \times 3 \times 4 \times 3 = 180$ configurations $\times$ 3 seeds $\times$ ~3-5 s each, about 30-45 min. Alternatively, tune one monotone knob at a time.

Freeze every configuration that passes, before any policy comparison, and run every player on all of them.

Limits of held-out checks

With one month and shared aggregates, held-out summaries are not fully independent evidence. Passing them makes the model more plausible but does not establish true demand, the real dispatch policy or expected real-world savings. A baseline mismatch may be tolerable for a method experiment, but it limits Hopkins-specific interpretation. Our stand-in baseline is not TransLoc's dispatcher: passing means plausible, not validated.

Do not validate by construction

Reproducing margins enforced by balancing is an integrity check. It is not behavioral validation. Matching calibrated waiting summaries is not a prediction. A different dispatch policy should be allowed to change completion, waiting and overflow outcomes; never force those historical outcomes on every policy.

Gate G4, baseline plausibility.

Required evidence: Compare compatible completion and hourly patterns plus wait/ride summaries, with source uncertainty.

Phase 6 checks: the calibration grid is logged; parameters are frozen before any comparison.

If it fails: Diagnose model/population mismatch; narrow claims.

PART II · BUILD · 11

Experiments and value metrics

Phase 7 runs the pre-registered scenario grid × players × paired seeds, plus the equivalent-fleet runs, and logs runtimes. Four outputs carry the value story:

Output	What it shows
Value ladder	For each scenario, five bars (N, A, B, C, H) of timely passengers served (the primary metric), with P90 wait and Lyft overflows alongside. The four gaps are the values of pooling, waiting, coordination and information.
Equivalent fleet	Rerun A with 5, 6 and 7 vans on the same stream. A claim of the form "C with 5 vans matches A with 6" is one anyone can grasp. Report in natural units (vans, Lyft trips, minutes); dollars appear only as an illustration with ASSUMED prices.
Where C wins	A heat map of C - A over load rho × window W.
A null result	Still a result. If H - A is small, no dispatcher can help much, and the lever is fleet and shift timing (T1, T9). That is defensible and useful.

Metrics

Metric	Definition and reporting rule
Primary: timely passengers served	Sum party sizes for completed requests satisfying both wait and ride limits. Report count and share of all requested passengers.
Request service rate	Completed requests / all generated requests. Also report timely completed requests / all requests.
Waiting and riding	Wait = pickup - arrival; ride = dropoff - pickup. Report quantiles among served riders with weighting stated. Pair with unserved metrics to expose selection effects.
Modeled overflow / abandonment	Separate counts and rates with all requests as denominator. Never automatically label these Lyft trips or real cancellations.
Service by area	Origin/destination-zone service rates, waits and sample sizes. Avoid conclusions from tiny groups.
Vehicle and solver performance	Occupied, empty and idle time; end-of-shift overtime; runtime percentiles, feasible-solution and fallback rates.

Paired comparisons

For each scenario and replicate, reuse the same request stream and exogenous environment across policies. Pre-generate randomness by request ID and time/location where appropriate; one shared seed is insufficient if policies consume random draws in different orders. Compute policy differences within each paired night.

Report mean differences and uncertainty intervals across independent replicate pairs within a scenario. Use a paired bootstrap or a justified alternative. Choose replication count from pilot variability and a team-defined practical effect size. Simulation intervals represent random variation within assumptions, not all model uncertainty.

Guardrails and policy selection

Enter the minimum meaningful gain and maximum acceptable service deterioration before the main experiment. If limits are hard in a deterministic model, compliant served trips should satisfy them by construction; emphasize service coverage and wait distributions too. Without justified scenario probabilities, show scenario-specific results rather than an arbitrary weighted average.

Gate G5, experimental reliability.
Required evidence: Paired repetitions; uncertainty intervals; runtime logs; scenario sensitivity.
Phase 7 checks: paired bootstrap confidence intervals.
If it fails: Increase justified repetitions or report uncertainty.

PART II · BUILD · 12

Showing it: replay viewer and recommendation

A live, game-style viewer is feasible. The recommended form is a "live replay": simulate headlessly, then play the event log in a browser. It is deterministic and cannot break mid-presentation, because nothing is solved live.

Screen	What it shows
1. Scenario preview ("level select")	A map with faint request pins colored by request time, plus van starting positions; an arrival histogram; a parameter card (vans, requests, rho, W, capacity, limits, seed); a picker for the left and right players, and a Play button.
2. Play	Synchronized side-by-side maps (for example A and C) with a clock (18:23:10); play/pause, 1×, 10× and 60× speeds, skip-to-end, and a scrub bar. Pins pulse on request, ring while waiting, and vanish at pickup. Vans show their load and route ahead, and overflows flash red. A live scoreboard per pane (served, waiting, onboard, overflow, running P90 wait) and an event ticker ("18:07 Van 3 picks up R12, pooled with R9").
3. End of level	Final scoreboards and this scenario's value ladder.

Tech, build order and stretch goal

Tech: static HTML + Leaflet + replay.json. Road geometry comes from the existing src/demo/osrm.py; straight-line motion is fine for L. This replaces the folium demo, which cannot sync panes or show scoreboards. Open it in Chrome or Safari (Brave Shields blocks the map tiles).

Build a minimal player early (Phase 4). Watching vans move is the fastest way to find engine bugs.

Stretch goal, only after G5: a "sandbox" mode with a Python backend: click to drop a request and both players react live.

Final figures and the recommendation package

Three visuals carry the story: the digitized April dashboard (evidence), the side-by-side replay (mechanism), and the value ladder (result). One figure each, minimal text.

Produce an aggregate audit plot, scenario map, wait-versus-service tradeoff, policy-by-scenario improvement table and runtime summary. Show losses and null results. Separate random variation from differences across fleet, timing, patience and travel assumptions.

Phase 8 delivers the polished viewer, the final figures, the recommendation, the value to TransLoc (chapter 5) and the value of the next extract (Reference E), as a deck, a report and the viewer.

Permitted conclusion: "Policy C improved timely service in these tested scenarios, under these assumptions."
Unsupported conclusion: "Hopkins will save this percentage of Lyft trips."

Gate G6, recommendation.
Required evidence: Fresh test runs for any selected rule; harms and failure conditions reported.
Headline check: every headline follows the discipline in chapter 5: "Under these tested scenarios and assumptions..."
If it fails: Present exploratory findings rather than a deployment recommendation.

PART III · REFERENCE · A

Data contracts and file locations

Phase 1-2 files below exist as built; Phase 3+ paths are still proposed interfaces. docs/evidence/, data/ and outputs/ hold material derived from the restricted PDF and stay git-ignored until data sharing is confirmed. Keep restricted source material and generated operational tables out of public commits according to repository policy. All records need a stable ID and version.

Path / row unit	Fields	Status and rule
data/raw/april/pNN.png One PDF page image	Original pixels (pdfimages -png)	Built (Phase 1). The input to every chart read.
data/processed/obs_<chart>.csv, merged into observations.csv One chart observation	obs_id, chart_id, page, period, date_or_hour, metric, value, lower, upper, unit, population, evidence_status, method, px_x, px_y, notes	Built: 482 rows. Source readings with their uncertainty.
data/processed/map_bubbles.csv One detected bubble	map_page, map_name, bubble_id, colour, digits_expected, px_x, px_y, near_edge	Built: 214 bubbles.
data/processed/ map_counts_readerA.csv, _readerB.csv, _final.csv One bubble read	bubble_id, count_read, lower, upper, confidence, status; final adds agreement and both reads	Built: 215 rows; the readers agree on 214 (one adjudicated).
data/processed/map_clusters.csv, .geojson; georef_fit.json One map cluster	bubble_id, view, lat, lon, pos_uncertainty_m, count_best, lower, upper; the georeference fits	Built. Positions ±17 m (campus) to ±57 m (city).
data/processed/zones_v0.csv, zone_points_v0.csv One zone; one zoned bubble point	zone_id, zone_name, lat, lon, radius_m, origins and destinations with bounds, net_d_minus_o; points add their zone and fence scaling	Built (Phase 2, v0): 16 zones (D-13). Replaces the proposed zones.csv and zone_counts.csv.
data/processed/tt_zones_v0.csv, tt_points_v0.csv One directed pair	origin, dest, seconds, meters, source, retrieval_date	Built (v0): OSRM free-flow (D-12); retain direction. The contract's time_band and model_version are still to add (Phase 3).

Path / row unit	Fields	Status and rule
data/synthetic/requests.parquet One request	request_id, scenario_id, replicate, service_date, arrival_s, origin_node, dest_node, party_size, patience_s	Proposed (Phase 3). Generator only. No service outcomes here.
outputs/runs/.../request_results.parquet One request x policy run	request_id, run_id, terminal_status, assigned_s, pickup_s, dropoff_s, vehicle_id, wait_s, ride_s, failure_reason	Proposed. Simulator only; absent outcomes are null, never zero.
outputs/runs/.../run_manifest.json One policy run	run_id, config_hash, scenario_id, seed_set, policy, code_version, solver_settings, input_versions	Proposed. Runner: supports exact reproduction.

Store vehicle schedules separately: vehicle_id, capacity, start_s, end_s, initial_node. Store event logs separately: event_id, simulation_time, event_type, vehicle_id, request_id, location and load. The event log is essential for debugging impossible results. The replay viewer reads it through a fixed replay.json schema (chapter 12).

Time convention: seconds since the chosen service-night start internally; America/New_York and ISO dates in metadata. Define after-midnight attribution explicitly (D-03: rides after 00:00 belong to the previous service night). Never mix minutes and seconds in model inputs.

Code, outputs and documents

Area	Code	Outputs and documents
Evidence (Phase 1)	src/evidence/: digitize.py, chart_pNN_*.py, map_bubbles.py, map_reconcile.py, georef.py, georef_qa.py, od_maps.py, maps_recreated.py	outputs/evidence/: *_overlay.png, *_recreated.png, od_maps.html, p02_p05_recreated.png. docs/evidence/: README.md, briefs/<chart>.md, brief_template.md, data_quality.md, reconciliation_ledger.md, map_count_reconciliation.md.
Insights (Phase 2)	src/insights/: t01_supply.py to t10_ledger.py, common.py, basemap.py	outputs/insights/tNN_*.png with CSV twins (t10 is CSV only); docs/evidence/insights/tNN.md.
Scenarios (Phase 2-3)	src/scenario/: zones.py, travel_times.py (OSRM table service, cached in data/raw/osrm_cache/); the generator follows in Phase 3	The zone and travel-time files above; request and fleet files and the scenario grid spec (Phase 3, proposed).
Simulator (proposed)	src/sim/ (engine, rules, rider, log, metrics, runner) and src/sim/dispatchers/ (nearest, greedy, batched, milp, hindsight)	Event logs and the run outputs in the table above.
Viewer (proposed)	src/viewer/ (index.html, player.js)	replay.json
Docs	docs/technical/build_team_guide.py (this guide)	docs/technical/team_config.v2.json; docs/technical/decision_log.md; docs/technical/architecture/ (sim_architecture.drawio, p1-p4 SVGs, README.md); docs/ai_use_log.md.

New dependencies: pandas, matplotlib, scipy, highspy. **Architecture diagrams:** docs/technical/architecture/README.md explains how to open, edit and export them (VSDX, PNG, PDF).

PART III · REFERENCE · B

Parameter register and decision log

Fill the companion **team_config.template.json**, then save an experiment-specific copy. A null value means unresolved, not zero. Numerical choices need a source or an assumption rationale in the decision log. Every required null must be resolved before a main run.

Config key	Unit / value to enter	Basis and owner
service.start_local / end_local	Local HH:MM; initially null	Data owner checks report coverage and defines service night.
service.wait_limit_s / ride_limit_s	Seconds; initially null	Team verifies applicable rules. Earlier 20/25 minute discussion is provisional.
fleet.schedule_path	Path to vehicle-level schedule; null	Simulation owner defines plausible fleet scenarios. Do not silently use all-services counts.
demand.daily_totals_path	Path to extracted daily totals; null	Data owner specifies request population and repeat-booking caveat.
demand.hourly_request_weights	Nonnegative vector summing to 1; null	Model owner supplies alternative assumptions; not copied as fact from hourly completions.
demand.party_size_probs	Discrete probabilities; null	Use compatible passenger/request ratio if available; a mean alone does not identify probabilities.
demand.patience_spec	Distribution, parameters, limits; null	Scenario assumption. Independent of policy for paired runs.
spatial.beta_per_min / scenario_path	Distance sensitivity; scenario table; null	Model owner varies gravity and time-direction assumptions.
travel.matrix_path / multiplier	Path; positive factor; null	Geography owner supplies road model and congestion cases.
simulation.pickup_dwell_s / dropoff_dwell_s	Seconds; null	Simulation owner: assumptions. Vehicle hold limit is not automatically average dwell.
dispatch.windows_s / urgent_margin_s	[30,60,120] pilot / null	Team selects windows and shared urgency margin.
solver.time_limit_s / max_new_per_route	Seconds / integer; null	Optimization owner sets based on benchmark runtime.
evaluation.replicates / seed_base	Positive integer / integer; null	Analysis owner sets after pilot variability and seed design.

Also decide: travel-time randomness, allowable reassignment, overflow trigger, shift-end rule, horizon clearing, scenario weights, service guardrails and primary comparison. Record these before experiments; do not bury them in code.

Config v2 and the decision log. The pilot defaults are in **docs/technical/team_config.v2.json** (status PILOT_DEFAULTS_NOT_FROZEN: freeze a copy per experiment before Phase 7). Its values carry evidence labels in **docs/technical/decision_log.md**, decisions D-01 to D-13: for example D-01 (maps count completed rides, DERIVED), D-04 (20 and 25 minute limits, PROVISIONAL) and D-08 (party size: mean DERIVED, shape ASSUMED). A new number enters both files together.

How to enter values

The companion template is **docs/technical/team_config.template.json**. Copy it to a named experiment configuration before filling it. JSON has no comments, so rationale belongs in a separate decision log. Paths in the template are proposed interfaces; input datasets still need to be produced.

```
Example decision-log record (illustrative only):
parameter: dispatch.urgent_margin_s
value: 30
evidence_status: ASSUMED
reason: pilot buffer before the next dispatch opportunity
alternatives: [0, 30, 60]
owner: TEAM_MEMBER
reviewer: TEAM_MEMBER
decision_date: YYYY-MM-DD
status: proposed
```

The value 30 above is an example of documentation, not a recommended safety margin. For observations, replace the rationale with a source page and extraction interval. For verified rules, record the official source and applicable date. Required unresolved values must cause configuration validation to fail.

Before launching a main experiment

Confirm that required fields are populated; probabilities sum to one; units and date conventions match; input paths and versions exist; every scenario has a rationale; baseline checks are recorded; policy objectives are fixed; random streams are paired; solver fallback works; output directories are unique; and the code/config/input versions are saved.

PART III · REFERENCE · C

Team workflow, agent plan and notebooks

Phases, each ending at its gate

Phase	What	Output	Gate	Est.
0 Align	Guide v2.0; architecture in draw.io -> VSDX, SVG, PNG; defaults in config v2 + decision log; start the AI-use log; the professor's comment	Guide PDF, .drawio + .vsdx, config v2	Every teammate can explain chapter 2 and the designs in chapters 7-9	2-3 days
1 Evidence (done)	Digitize all 12 charts: CSVs with intervals, overlays, recreated charts, 12 briefs, digital O/D maps, reconciliation ledger	data/processed/..., docs/evidence/...	G1 (passed 2026-09-29)	~1 week
2 Insights (done)	T1-T10, plus zones v0 and an OSRM table (needed for T6 and T8)	10 figures + briefs (mid-semester material)	Every number labelled and traceable	~1 week

Phase	What	Output	Gate	Est.
3 World	Zones v1, travel times, generator (gravity + IPF + sampling); base M frozen and seeded; S0-L defined; scenario grid pre-registered	Request and fleet files, grid spec	Margins reproduced; seeds reproducible	~1 week
4 Engine	Event engine + referee + rider model + N and A; minimal replay player; run S0 -> S1 -> S2 -> M	Event logs, accounting	G2 hand-checks; determinism test	~1.5 weeks
5 Players	B; C (route-selection MILP, HiGHS); H (node-arc, small instances, bracketed); shared fallback	Dispatcher plug-ins	G3 parity; C equals brute force on S1	~1.5 weeks
6 Calibrate	Calibration-lite at L with A; freeze	Sim-vs-April figure	G4	~0.5 week
7 Experiments	Grid × players × paired seeds; equivalent fleet; runtimes	Ladder, heat map, CIs	G5	~1 week
8 Show	Polished viewer, final figures, recommendation, TransLoc value, value of the next extract	Deck, report, viewer	G6	~1 week

Checkpoint: the end of Phase 4 makes a strong mid-semester update: April fully used, a base scenario, a working engine, and a first replay.

Work packages

Work package / suggested owner	Deliverable	Completion gate
1. Data steward (Phases 1-2) Name: _____	Observation tables, dictionary, reconciliation ledger; aggregate diagnostics (T1-T10) in parallel.	G1 passes or unresolved items have explicit downstream scenarios.
2. Geography + demand (Phase 3) Name: _____	Zones, road matrix, scenario generator and assumptions.	Reproducible streams; feasible geography; margins and population interpretation documented.
3. Simulation engineer (Phase 4) Name: _____	Event engine, accounting, greedy policies, logs; the minimal replay player.	G2 and G3 pass on hand-checkable cases.
4. Optimization + evaluation (Phases 5-8) Name: _____	Candidate-route optimizer and H, calibration-lite, pilot experiments, sensitivity and paired analysis.	G4-G6 assessed; all limitations reported.

Roles can be shared, but each artifact needs one accountable owner and a different reviewer. Work packages 1 and 2 can progress together after the source manifest; optimization can use tiny artificial instances while the demand model is being built.

Agent plan

Rule: agents take tasks that are well specified, independently checkable, and write their own output files. The main thread (the team's primary working session) keeps design authority: engine semantics, the MILP, calibration choices, and interpretation, that is, everything the team must defend in the Q&A.

First one by hand, then agents: the shared helper and one worked example (src/evidence/digitize.py on p11, checked against the Apr 17 anchors) come first; agents replicate that pattern. **Each agent gets** a self-contained prompt, input paths, output paths only it writes, acceptance checks, and "do not edit shared files; list any dependencies you need". Mechanical tasks run on Sonnet, in the background and in parallel; code-writing agents run in a git worktree. **The test author is not the implementer:** S0 and S1 expected outcomes come from an agent working independently of the engine code. Every result is checked against its acceptance checks before merging and logged in docs/ai_use_log.md (tool, purpose, how used, how verified).

Wave	Agent	Task	Acceptance check
0	G1	Port the approved v2.0 content into docs/technical/build_team_guide.py (reportlab layout only)	Section map 1:1; builds cleanly
0	G2	Author the 4-page .drawio (overview, players grid, engine loop, data flow) from the spec; export	Opens in diagrams.net; VSDX opens in Visio for the web
1	E1	p6 + p9 (hourly bars)	27.6 and 5,098 within ±1%
1	E2	p7 + p8 (status lines, passengers)	1,388 ±1%; daily completed sum ~ hourly sum ±3%
1	E3	p12 (ride quantiles)	Apr 17 anchors ±10 s
1	E4	p10 (PowerBI)	Van + Lyft ~ 44,374 ±2%
1	E5	Blind second read of every map cluster (p2-p5)	Disagreement list vs the main read
2	I1-I5	T2, T4, T7, T9, T10 draft	Numbers trace to observation IDs
2	W1	OSRM table matrix + cached route geometries	Sanity checks; cache hits on rerun
4	V1	Replay player from a fixed replay.json schema + a mock log	Plays, scrubs, 60× on the mock log
4	T1	Hand-check cases for S0/S1	Expected values derived by hand or enumeration

Agent IDs are labels only: agent G1 is not gate G1, and agent T1 is not task T1. **Main thread:** the p11 worked example; maps p2-p5 (georeferencing and first read); reviewing all briefs; T1, T3, T5, T6, T8; signing T10 at G1; zones; the generator; the engine; B, C, H; calibration; experiments; interpretation.

Suggested notebooks (interfaces to later implementation)

00_source_audit.ipynb	observations + reconciliation
01_aggregate_diagnostics.ipynb	profiles + anomalies
02_zones_and_travel.ipynb	geography + directed costs
03_scenario_generator.ipynb	demand + construction checks
04_simulator_checks.ipynb	invariant and example checks
05_policy_pilot.ipynb	timing + assignment contrasts
06_main_experiments.ipynb	paired runs + metrics
07_sensitivity_and_report.ipynb	assumptions + recommendation

First team meeting: decisions to fill in

Primary comparison: _____ Owner: _____

Service rules source: _____ Reviewer: _____

Initial geographic extent: _____

Required pilot scenarios: _____
Practical improvement / harm thresholds: _____
Pilot review date: _____ Main run freeze date: _____

Open items from plan v2
Mid-semester update date: _____ Owners for Phases 3-8: _____
Repo visibility: may the digitized April data be committed? _____
Sources for van capacity and service limits (or keep them ASSUMED): _____

Prof. Kearsley's comment: received 2026-09-29. Assess the quality of the data you have; getting an idea of what you can extract is important. He called it "very good data ... chock full of exploitable information" and endorsed the direction ("gradient pointing in the right direction"). Location analysis and dispatch-scheme comparison are each project-sized, and related. Our first answer is the data-quality scorecard (chapter 4; docs/evidence/data_quality.md).

Prof. Kearsley's first note (on the aggregate data). It is rare to get exactly the data you want; the aggregate data is reasonable to good; from it one can gather "sufficient statistics" to "pull apart operating regimes" and, if we choose, build synthetic data "consistent with the aggregate". Chapter 1's stages follow that sequence: statistics and regimes first (stages 1-2, T1-T10), then synthetic nights constrained by them (stage 3).

PART III · REFERENCE · D

Sources, verification, risks and AI disclosure

Sources and scope of verification

Operational source to extract: *HW Night Ride Data April 2026.pdf*, provided to the team; it is restricted, so it stays out of git. This guide specifies an extraction workflow; it does not independently certify chart values or service targets. Its April numbers come from the Phase 1-2 evidence pack, each with its evidence label.

Evidence pack (Phases 1-2, indexed by docs/evidence/README.md): `data_quality.md`, the six-dimension scorecard for all 12 charts; `reconciliation_ledger.md`, every cross-chart identity with its residual, the G1 verdict and its 7 carried caveats; `map_count_reconciliation.md` (two blind readers); `briefs/` and `insights/`.

Project context reviewed: repository README.md, the project-direction discussion through 22 September 2026, the approved plan v2 of 29 September 2026, and the evidence pack. The README still describes potential trip-level acquisition; this guide adopts the user-confirmed constraint that no additional operational data will arrive.

Methods: the route-selection formulation, experimental controls and proposed schemas in this guide are a design specification. They are not a literature review or a claim of methodological novelty. Add primary research and verified tool documentation during implementation, before making publication or provider-capability claims.

Known limits, operational risks and data governance

Limit or risk	Response
The vans chart is "All services"	Resolve or scenario-test all-services versus Homewood coverage (chapter 6).
The maps show served trips only	Label the population interpretation of every scenario (chapter 7, step 3).
Our baseline is a stand-in, not TransLoc's dispatcher	Calibration passing means plausible, not validated (chapter 10).

Limit or risk	Response
M understates density	Keep one L check (chapter 7).
Capacity and service limits are ASSUMED	Vary them as scenarios; source them or keep ASSUMED (open item in C).
The public OSRM server is for light use only	Cache, use one table call, and fall back to a local OSRM if needed.
H may not close	Report the bracket [incumbent, bound] (chapter 9).
MILP solve time	Logged; slower than W counts as operationally infeasible (chapter 9).
Data governance: may the digitized April data be committed?	Depends on repo visibility; confirm, and keep it untracked until then.

If we fall behind: the cut list

Use gates rather than an assumed semester calendar. Drop optional extras first (fine spatial detail, optional place enrichment); then cut in this order:

1. Sandbox mode.
2. L experiments beyond one check.
3. H on M (keep it on S1/S2).
4. Fancy viewer screens (keep the minimal replay).
5. Extra grid dimensions, such as extra policy windows.

Never cut: G1 quality, G2 hand-checks, pairing, the MILP verified on toy instances, and honest limits. Keep correct pooling, fair comparisons, burst sensitivity where relevant, accounting checks and honest uncertainty reporting.

Research disclosure (AI use): this guide was drafted with AI assistance. Team members should verify source readings, understand and review the model, and document AI use according to the course requirements. The deliverable has no simulated performance findings yet. An AI-use log is kept at **docs/ai_use_log.md**: each entry records the tool, the purpose, how the output was incorporated, and how it was verified, as the syllabus requires.

Next concrete milestone: Phases 0-2 are done (this guide, the architecture diagrams, all 12 charts digitized with their briefs, T1-T10, and G1 passed). Next is Phase 3: zones v1, travel times, the generator, base scenario M frozen and seeded, S0 to L defined and the scenario grid pre-registered; then one hand-verified pooled-route example (S0, S1). Once those exist, the team can build with a shared understanding of what is measured and what is assumed.

PART III · REFERENCE · E

What more is possible: the value of the next extract

Prof. Kearsley's first note is right: it is rare to get exactly the data you want, and this project is designed to finish on the data we have. This list is for Hopkins and TransLoc. Each item is ranked by cost to provide, and each names the assumption it would retire and the result it would sharpen: a value-of-information statement, not a complaint.

Extract (cheapest first)	Cost	Retires (ASSUMED or caveat -> OBSERVED)	Sharpens
1. Cancel reason counts: rider, timeout, Lyft handoff, rebooked	A dashboard filter	What "Canceled" includes (G1 caveat); requests as demand	Base load: 30 ("as served") or 60 ("as requested") requests, a 2x range on rho; the patience model; the Lyft overflow share
2. Vehicles chart filtered to Homewood Night Ride	A dashboard filter	The "All services" filter (G1 caveat)	Per-van productivity: T1, T5 and T9 are bounds today
3. Cancellations by hour (the p9 chart for cancelled requests)	A dashboard filter	Hourly request weights (null in config v2)	The true size of the 6 pm peak; shift design (T9)
4. Waits by hour (the p11 chart split by hour)	A dashboard filter	A whole-night wait band used for an hourly window	Calibration at the right grain (G4)
5. Anonymized request log: timestamp, O and D zone, status, party size	One extract	The O-D joint distribution (IPF), the arrival process, patience, the party-size shape	Synthetic streams become replayable ones; the generator becomes OBSERVED
6. Vehicle summaries: occupancy and deadhead by hour	One extract	Occupancy lower bounds (T5); the detour split (T6)	Direct validation of pooling and routing in the simulator

Sources: the last section of docs/evidence/data_quality.md, and the nulls in config v2 (Reference B).

PART III · REFERENCE · F

Revision history

Each version is archived as a PDF in `output/pdf/archive/` and as a builder snapshot in `docs/technical/guide_history/`; `docs/technical/guide_changelog.md` holds the details. To change the guide: bump VERSION and DATE, add a row here and an entry in the change log, build, then snapshot the builder.

Version	Date	What changed	Why
v1.0	22 Sep 2026	First team guide: pipeline, policies A-C, data contracts, gates G1-G6, calibration, experiments (<code>build_team_guide_v1.py</code>)	Plan of record after the proposal
v2.0	29 Sep 2026	Reordered into Think, Build, Reference; April-shaped base scenario and scale ladder; players N and H; calibration-lite; replay viewer; a gate box closing each Part II chapter	Plan v2, after the aggregate-only scope (28 Sep)
v2.0 refresh	30 Sep 2026	Chapter 4 filled with the confirmed Phase 1-2 results; G1 marked passed; data-quality scorecard; Reference C records Prof. Kearsley's feedback (in place, same version number)	Phases 1-2 complete
v2.1	30 Sep 2026	Chapter 1: the stage flow shows "Analyze and mine" (T1-T10) as its own stage, plus a directions table (lines 0-3); the cover takeaway mentions mining. Chapter 4: the Apr 17 profile. Chapter 5: item 5 becomes the value of the next extract, with the detail in the new Reference E. New Reference F. Reference C: Prof. Kearsley's first note. Layout fixes.	Team review of v2.0; Prof. Kearsley's two notes

PART III · REFERENCE · V1 TO V2

What moved where

Every v1.0 section appears exactly once in v2.0; this page is the Phase 0 check. Page numbers are v2 pages.

v1	v1 section	v2 location
p1	Title and subtitle; research question box; version line; scope paragraph; "What this document is"; "How teammates should use it"; navigation line	Cover, p. 1 (updated; navigation is now the Parts I-III table)
p2	Pipeline box; operational decision; what counts as success; scope table; four evidence labels; external-geography note	Ch 1, p. 2 (pipeline and scope table updated)
p3	Policy table (A, B, C)	Ch 9, p. 14: merged into the players table
p3	B versus A comparisons; pilot settings; rules held fixed; urgent-request behavior; first pilot questions; gate box	Ch 9, p. 14 (the gate box is the design gate before coding C, p. 15)
p4	Source manifest paragraph; chart-family table; extraction procedure (merged with the bar, line and map methods); reconciliation ledger; "Do not force equality" box	Ch 6, p. 8
p5	Interfaces note; contracts table (raw-pixel and map-cluster rows added); vehicle schedules and event logs; time convention	Ref A, p. 20
p6	Register intro; config-key table; "Also decide"	Ref B, p. 22
p7	Steps 1-5 (steps 3 and 4 updated for the base scenario); "Keep input and outcome apart" box	Ch 7, p. 10
p8	Event loop and event ordering; request states; outcome definitions; commitments and end of service; constraints; accounting-identity box	Ch 8, p. 12
p9	Route-option MILP; notation and model; route options; objective and fairness; computation; prototype warning	Ch 9, p. 14
p10	Gate table, rows G1 to G6	Gate boxes: G1 p. 10, G2 p. 14, G3 p. 16, G4 p. 17, G5 p. 19, G6 p. 20
p10	Calibration protocol; limits of held-out checks; do not validate by construction	Ch 10, p. 16
p11	Metrics table; paired comparisons; guardrails and policy selection	Ch 11, p. 18
p11	Final figures; permitted and unsupported conclusion box	Ch 12, p. 19
p12	Work packages (phases added); roles; suggested notebooks; first-meeting worksheet (open items added)	Ref C, p. 23
p12	"Use gates rather than an assumed semester calendar" and the cut advice	Ref D, p. 26: merged with the cut list
p13	How to enter values; example decision-log record; before launching a main experiment	Ref B, p. 22
p13	Sources and scope of verification; research disclosure; next-milestone box (updated)	Ref D, p. 26

New in v2.0 (from plan v2): chapters 2-5, with the confirmed Phase 1-2 results and the data-quality scorecard in chapter 4; the base scenario and scale ladder (7); the game table and code shape (8); players N and H with the ladder of gaps (9); calibration-lite (10); the value outputs (11); the replay viewer (12); code locations (A); config v2 defaults (B); the phase and agent plans (C); limits, risks and the cut list (D); a gate box at the end of every Part II chapter.